Algorithm Analysis - US14

Introduction

This document analyzes the **time complexity** of the algorithms implemented in **US14**. The primary algorithms involved are `transitiveClosure`, `buildMaintenanceRoute`, `findEulerRoute`, `firstVertexWithDeg`, `dfsMatrix`, and `isBridge`. These methods are used to analyze graph structures, compute the transitive closure, identify Eulerian routes, and check for specific graph properties like bridges. The analysis will focus on determining the worst-case time complexity using Big O notation, helping us understand how these algorithms perform as the size of the graph increases.

---

1. `transitiveClosure` Method

The `transitiveClosure` method computes the transitive closure of a graph represented by the adjacency matrix `M`. The transitive closure is a matrix `F` that indicates whether there is a path between any two vertices. This is done using a variant of the **Floyd-Warshall algorithm**.

`transitiveClosure` Pseudocode Analysis

Operation	Approx. Elementary Operations	Big-O Complexity
`boolean[][] F := M`	1A	**O(1)**
for `k := 1` to `n`	(n+1)A + (n+1)C	**O(n)**
--- **for** `i := 1` to `n`	$(n+1)^2$A + $(n+1)^2$C	**O(n^2)**
---- `if F[i, k] := 1` then	n^2C	**O(n^2)**
------- **for** `j := 1` to `n`	$(n+1)^3$A + $(n+1)^3$C	**O(n^3)**
--------- `F[i][j] := F[i][j] or F[k][j]`	n^3A	**O(n^3)**
`return F`	1R	**O(1)**
Total		**O(max(O(n^3)+O(n^2)+...) = O(n^3)**

Explanation:

1. **Initialization**: The matrix `F` is initialized to the adjacency matrix `M`, which takes **O(1)**.
2. **Triple Nested Loops**: The algorithm uses three nested loops to propagate connectivity between all pairs of vertices. Each of these loops runs **n** times, resulting in **O(n^3)** complexity.

Total Complexity:

The time complexity of the `transitiveClosure` method is **O(n^3)** due to the nested loops that examine every combination of vertex pairs.

---

2. `buildMaintenanceRoute` Method

The `buildMaintenanceRoute` method is used to check the connectivity and properties of a graph, and it utilizes the transitive closure and other methods to validate graph conditions. It verifies that all vertices are connected, and it checks the degrees of the vertices to see if an Eulerian circuit or path is possible.

`buildMaintenanceRoute` Pseudocode Analysis

Operation	Big-O Complexity
`if adj == null` then `return false`	**O(1)**
`degree[] := 0`	**O(n)**
for `i := 1 to n`	**O(n)**
---- **for** `j := i+1 to n`	**O(n²)**
------- `if adj[i,j] then degree[i] := degree[i] + 1 ; degree[j] := degree[j] + 1 ; edges += 1`	**O(n²)**
`if edges == 0` then `return false`	**O(1)**
`ConnectivityUtils.transitiveClosure(adj)`	**O(n³)**
for `i := 1 to n-1`	**O(n)**
---- `if degree[i] == 0` then `return false`	**O(n)**
for `j := 1 to n-1`	**O(n²)**
---- `if adj[i][j] = 0 && degree[j] > 0`	**O(n²)**
------- `return false`	**O(1)**
`int odd = 0`	**O(1)**
for `d in degree`	**O(n)**
---- `if d % 2 != 0 then odd += 1`	**O(n)**
`if odd == 0` then `return true`	**O(1)**
`else if odd == 2` then `return true`	**O(1)**
`else`	**O(1)**
`return false`	**O(1)**
Total	**O(max(O(n³)+O(n²)+...) = O(n³)**

Explanation:

1. **Graph Validation**: The initial steps check for null inputs and validate the graph's degree, which takes **O(n²)**.
2. **Transitive Closure**: The call to `ConnectivityUtils.transitiveClosure(adj)` has **O(n³)** complexity.
3. **Degree Checks**: Checking the degree of nodes and ensuring certain properties (like having an Eulerian path) is done in **O(n²)** time.

Total Complexity:

The time complexity of the `buildMaintenanceRoute` method is dominated by the transitive closure computation, resulting in an overall complexity of **O(n³)**.

---

3. `findEulerRoute` Method

The `findEulerRoute` method tries to find an Eulerian route in a graph. It checks the degree of the vertices and uses the `buildMaintenanceRoute` method to validate the graph's properties before finding the Eulerian path or circuit.

`findEulerRoute` Pseudocode Analysis

Operation	Big-O Complexity
`if adj == null`	**O(1)**
`degree[] = 0; edges = 0; k = 0`	**O(n)**
for `i := 1 to n`	**O(n)**
---- **for** `j := i+1 to n`	**O(n²)**
------- `if adj[i][j] == 1` then	**O(n²)**
-------- `degree[i] = degree[i] + 1; degree[j] = degree[j] + 1; edges = edges + 1`	**O(n²)**
`if edges == 0` then `return null`	**O(1)**
`if buildMaintenanceRoute(adj) == 1 then`	**O(n³)**
`int start = firstVertexWithDeg(degree)`	**O(n)**
for `v := 1 to n`	**O(n)**
---- `if degree[v] % 2 != 0 then start = v; break;`	**O(n)**
`trail[]`	**O(1)**
`int u = start;`	**O(1)**
while `edges > 0`	**O(n)**
---- **for** `v := 1 to n`	**O(n²)**
------- `if adj[u][v] = 1 and isBridge(u, v, adj, degree) = 1`	**O(n^4)**
-------- `next = v; break;`	**O(n^4)**
Total	**O(max(O(n^4)+O(n^2)+...) = O(n^4)**

Explanation:

1. **Graph Initialization**: Similar to previous methods, the degree of vertices and edges are counted in **O(n^2)** time.
2. **Eulerian Path Finding**: The method tries to find the Eulerian path by checking vertices' degrees and calling `isBridge`, which is **O(n^2)**. The **while loop** operates on edges, and checking for bridges increases complexity to **O(n^4)**.

Total Complexity:

The overall time complexity of the `findEulerRoute` method is **O(n^4)** due to the complexity of the `isBridge` method within the loop.

---

4. `firstVertexWithDeg` Method

The `firstVertexWithDeg` method finds the first vertex with a degree greater than zero, which is useful in the context of Eulerian path algorithms.

`firstVertexWithDeg` Pseudocode Analysis

Operation	Approx. Elementary Operations	Big-O Complexity
`for i := 1 to n`	**O(n)**	**O(n)**
`---- if deg[i] > 0 then return i;`	**O(n)**	**O(n)**
`write "Graph has no edges"`	**O(1)**	**O(1)**
Total		**O(max(O(n)+O(1)+...) = O(n)**

Explanation:

The method iterates through the degree array and returns the first vertex with a non-zero degree, which is an **O(n)** operation.

Total Complexity:

The time complexity of the `firstVertexWithDeg` method is **O(n)**.

---

5. `dfsMatrix` Method

The `dfsMatrix` method performs a depth-first search (DFS) on the graph represented by an adjacency matrix.

`dfsMatrix` Pseudocode Analysis

Operation	Approx. Elementary Operations	Big-O Complexity
`vis[v] = 1;`	**O(1)**	**O(1)**
for `u := 1 to n`	**O(n²)**	**O(n²)**
`---- if adj[v][u] == 1 and vis[u] == 0 then dfsMatrix(u, adj, vis)`	**O(n²)**	**O(n²)**
Total		**O(max(O(n²)+O(1)+...) = O(n²)**

Explanation:

The DFS function recursively visits all unvisited vertices, and the matrix representation of the graph requires **O(n²)** to check all edges.

Total Complexity:

The time complexity of the `dfsMatrix` method is **O(n²)**.

---

6. `isBridge` Method

The `isBridge` method checks if a particular edge is a bridge in the graph.

`isBridge` Pseudocode Analysis

Operation	Big-O Complexity
`if deg[u] == 1 then return false;`	**O(1)**
`adj[u][v] = adj[v][u] = false;`	**O(1)**
`boolean[] vis = new boolean[adj.length];`	**O(n)**
`dfsMatrix(u, adj, vis);`	**O(n^2)**
`boolean disconnected = not vis[v];`	**O(1)**
`adj[u][v] = adj[v][u] = true;`	**O(1)**
Total	**O(max(O(n^2)+O(1)+...) = O(n^2)**

Explanation:

The method temporarily removes an edge and performs a DFS to check if the graph is still connected. This DFS operation takes **O(n^2)** time.

Total Complexity:

The time complexity of the `isBridge` method is **O(n^2)**.

Conclusion

The time complexities of the algorithms in **US14** range from **O(n)** to **O(n^4)**, with the most complex methods being `findEulerRoute` and `isBridge`, which both have **O(n^4)** time complexity due to the use of depth-first search (DFS) and bridge-checking logic. The **`transitiveClosure`** and **`buildMaintenanceRoute`** methods have **O(n^3)** complexity, while **`dfsMatrix`** and **`firstVertexWithDeg`** are **O(n^2)** and **O(n)**, respectively. These complexities highlight the challenges in optimizing graph-based algorithms for larger datasets.